

Lab 10: Implement SQLite CRUD and Orders Safely

Maps to: A3, A4, A5

Connect product, customer and order operations to parameterized SQL and atomic transactions.

AI Vibe Coding Assets

- ai_vibe.py - OpenAI Python SDK runner
- mock-data.json - synthetic request and test context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - first-run and refinement prompts
- working-files/generated/ - isolated AI output for review

First-Run Prompt

Implement parameterized SQLite repository functions for product/customer CRUD and atomic order creation. Validate stock inside the same transaction, insert order items, decrement inventory or roll back fully, use bounded pagination and an allow-list for PATCH columns. Add CRUD, insufficient-stock and injection-shaped-input tests.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
pytest -q
```

Detailed procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 3: Run `python ai_vibe.py --dry-run` to validate and preview the exact SDK request without using API credits.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 4: Set `OPENAI_API_KEY` in the shell and choose an available `OPENAI_MODEL`; never paste either value into source files.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 5: Run `python ai_vibe.py` and inspect the typed `CodeBundle` written under `working-files/generated/`.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 7: Implement `create_product` with placeholders and return the inserted row.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 8: Implement `list_products` with bounded limit/offset and optional status filter.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 9: Implement `create_customer` and reject duplicate email addresses.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 10: Implement `create_order` as one transaction that checks stock, inserts order rows and decrements stock.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 11: Implement dynamic PATCH SQL from a validated allow-list of fields.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 12: Implement `delete_product` and inspect rowcount.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 13: Run CRUD and injection-shaped-input tests against a temporary database.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 14: Run `pytest -q` and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 15: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Acceptance criteria

Catalog/CRM CRUD and order tests pass, insufficient stock rolls back fully, missing IDs map to 404 and SQL values are parameter-bound.